

How ECR Targets Chaos

A walk through the method, its geometry and its failure modes, using the Lorenz system and numbers from a working replication.

Chaos control has two separate problems. *Stabilising* an unstable orbit once you are next to it is the easy one. *Getting there* is the expensive one — and that is the problem ECR solves, using nothing but recorded input and output.

01 The waiting problem

OGY-style control can only act when the trajectory happens to wander into a small neighbourhood of the target orbit. Until then you wait. On the Lorenz section used throughout this page, only **6% of crossings** land inside that neighbourhood, and because the dynamics are chaotic, being close once tells you nothing about being close next time.

Here is one real starting point, run twice with identical initial conditions. Left to itself, the trajectory takes **97 crossings** to enter the target region — and it teases, passing within 0.37 at step 14 and 0.56 at step 19 before being flung back out. With ECR-III steering, the same start is captured in **3**.

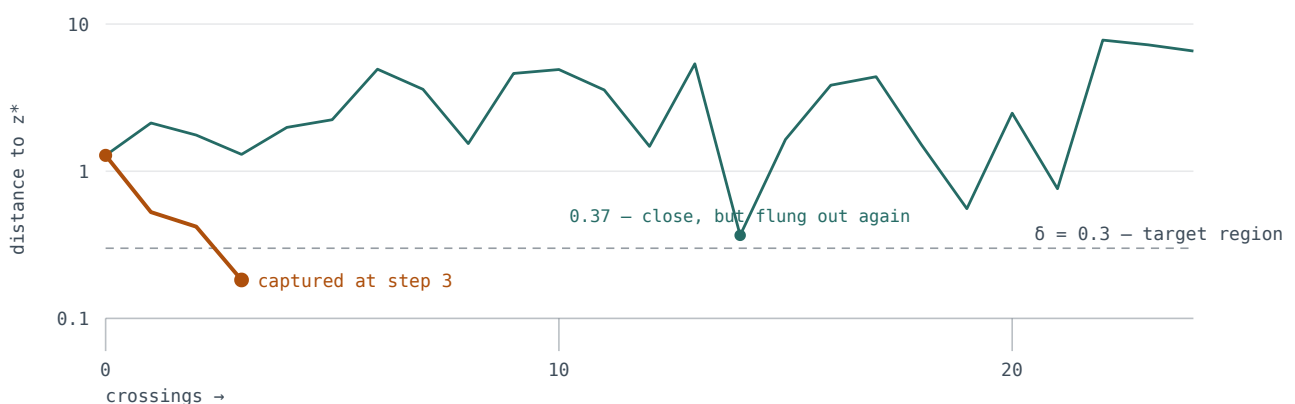


Fig 1. Same initial condition, two runs; the uncontrolled run continues past the axis, entering the region at crossing 97. Teal = no control, the parameters left at nominal. Orange = ECR-III. The vertical axis is logarithmic; the dashed line is the target region. Averaged over 100 starts these become 15.5 steps and 4.3 steps.

Targeting is what closes that gap. The trick is not a better controller near the orbit — it is knowing, from far away, *which small parameter nudge now puts*

you somewhere useful next crossing.

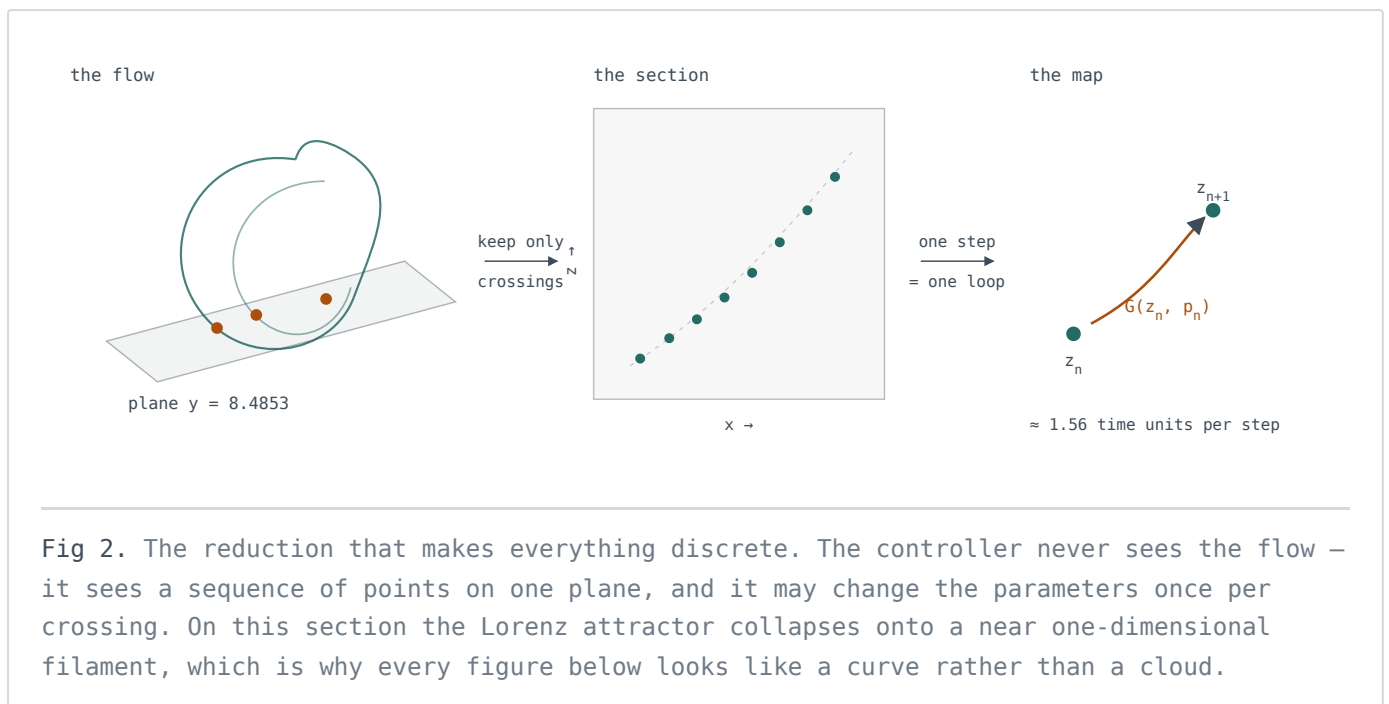
02 The stage: from a flow to a map

The Lorenz system is a continuous flow in three dimensions,

$$\begin{aligned} dx/dt &= \sigma(y - x) & dy/dt &= \rho x - y - xz & dz/dt &= xy - \beta z \end{aligned}$$

Control theory is far more comfortable with discrete maps, so we watch the flow only where it punches through a fixed plane — a **Poincaré surface of section**. Here the plane is $y = 8.4853$, crossed downwards. Every crossing gives one observed state, the pair $z_n = (x, z)$ at that instant, and the flow between two crossings becomes a single map step:

$$z_{n+1} = G(z_n, p_n)$$



03 The target, and why it resists

The target z^* is a fixed point of that map: a state which, at nominal parameters, returns exactly to itself one crossing later. It corresponds to a period-1 unstable periodic orbit of the flow — a loop the system would repeat forever if only it could be balanced on it.

$$\begin{aligned} z^* &= (14.2522, 39.7867) && \text{eigenvalues of } dG/dz: \\ \lambda_1 &= 4.71, \quad \lambda_2 \approx 10^{-9} \end{aligned}$$

Those two numbers tell the whole story of the difficulty. One direction is **expanding by a factor of 4.7 every crossing** — any error is five times worse one step later. The other is contracting so hard that the map is effectively one-dimensional; the attractor is a filament, not a sheet.

What can we do about it? Only nudge the parameters, and only slightly. Each of σ, ρ, β may move within 3% of nominal — the set Π . That budget is worth more than it sounds: it can displace the next crossing by about $(2.3, 4.2)$, roughly ten times the radius of the target region. *Reachability is not the problem. Knowing which nudge to use is.*

Why not just invert the map? Because we have refused to know it. ECR assumes no equations, no Jacobians, no manifolds — only the ability to run the experiment, measure the state and set the parameters. Everything below is built from that.

04 The idea: regions that are one step from home

Start from the destination and work outwards.

T_0 , the target region, is the set of states that are already inside the δ -ball *and* can be kept inside it by some admissible parameter. This is where the local controller works.

T_1 is the set of states which are not in T_0 , but from which some admissible parameter lands you *in* T_0 next crossing. T_2 is one step from T_1 , and so on. The regions are disjoint, and membership carries a promise:

If the current state is in T_i , then a sequence of i admissible nudges puts it in the target region. Not "eventually" — in i steps.

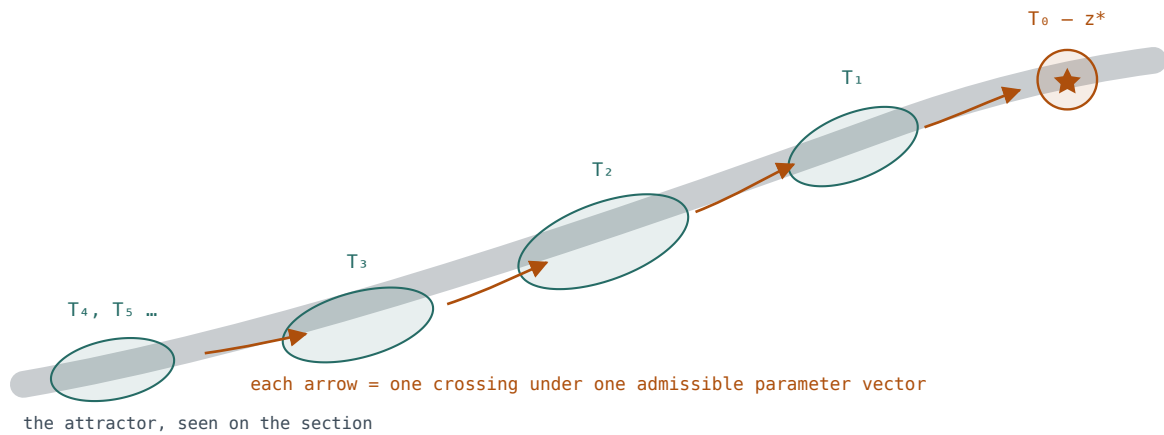


Fig 3. The nested structure that gives the method its name. The regions are not concentric rings around the target – they are stretched along the attractor, because that is where the dynamics can actually take you. A state in T_3 is three nudges from home; a state outside every region gets nominal parameters and is left alone until it drifts into one.

This also explains the name. OGY activates on one small region; ECR *extends* the activation region to the union of $T_0 \dots T_K$, so the controller has something useful to do far earlier.

05 Learning the regions from data alone

The definitions above quantify over "some admissible parameter" — an existence claim about a map we have refused to know. The resolution is the whole trick of the method: **run the experiment and let the data witness the claim.**

The plant is run from many initial conditions while, at every crossing, a parameter vector is drawn at random from Π . Each step contributes one transition triple:

(z_n, p_n, z_{n+1}) — where I was, what I did,
where I ended up

Now the labelling. For T_0 the test is direct: was the state inside the ball, and did it stay inside? If yes, the parameter *that was actually applied* is the proof that an admissible one exists, and it becomes the training target of that region's network.

For the outer regions there is a chicken-and-egg problem: to know whether a successor is in T_{i-1} , you need to know T_{i-1} — which is itself only known

through data. ECR breaks it by building the regions bottom-up and using **the previously trained network as the membership test**: feed it the successor, and if it returns a parameter inside Π , the successor is recognised.

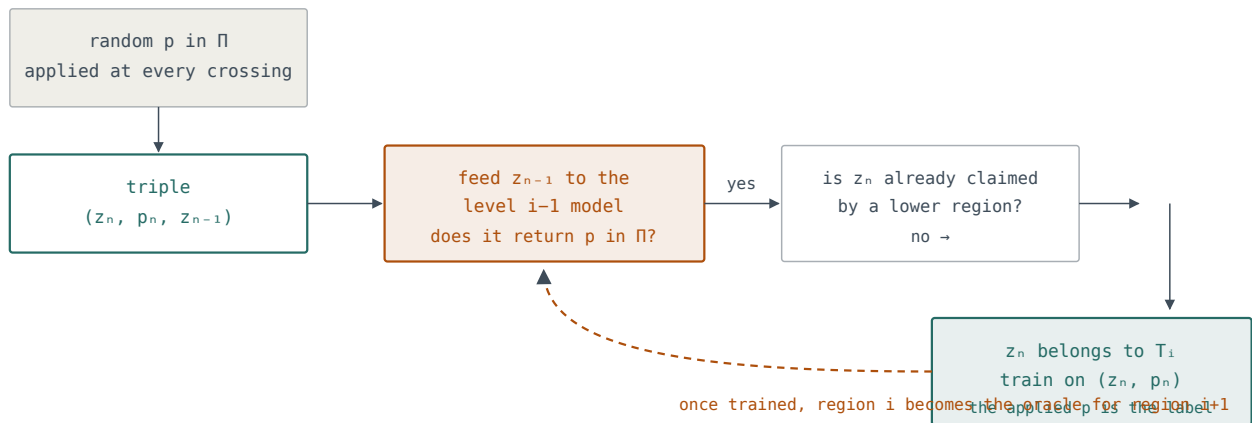
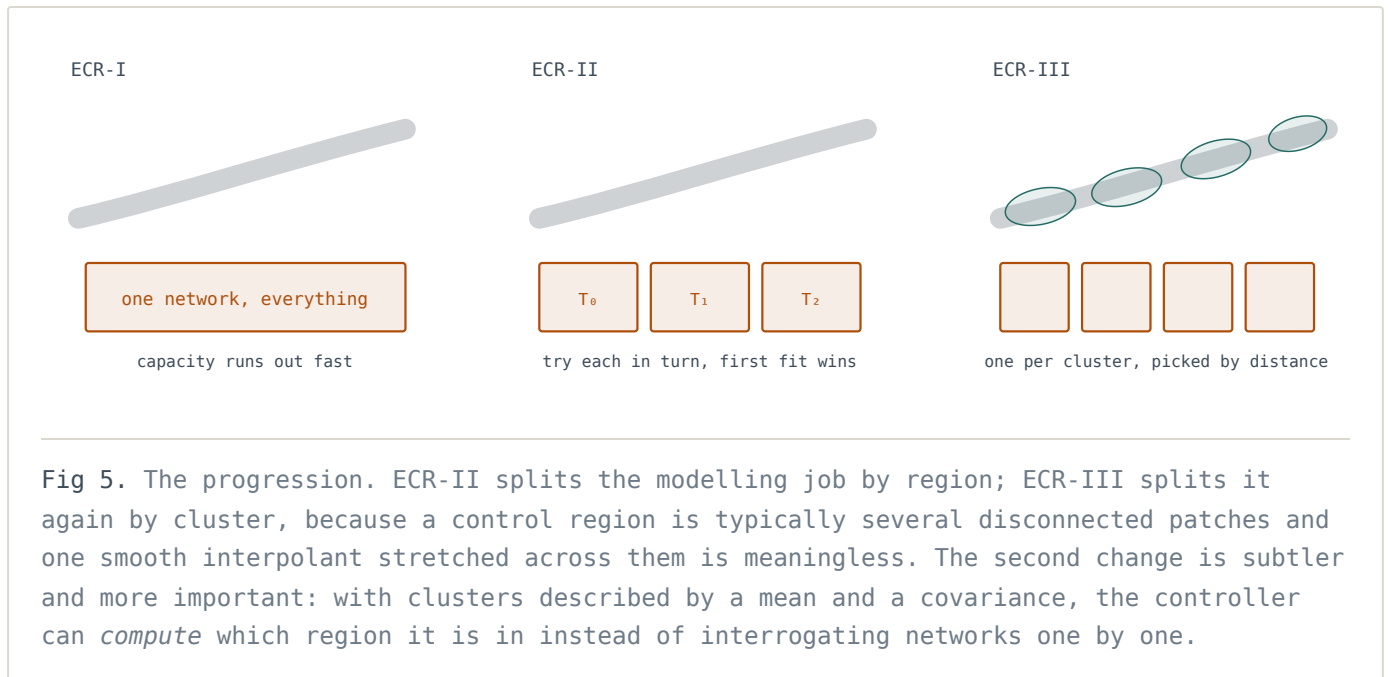


Fig 4. Region extraction. No system equations are consulted anywhere in this loop – the only oracle is a network trained on earlier data. On a 6800-triple Lorenz run this produced T_0 with 37 points, T_1 with 236, T_2 with 1369, then thinner outer regions, stopping at T_5 .

Why is T_0 so small? Only 6% of crossings land in the ball, and of those, only a minority happen to receive a parameter that keeps them there — one random draw per visit. That scarcity is real and it matters: the local controller is the part of the model trained on the least data, which is exactly why its behaviour after capture deserves separate scrutiny (§9).

06 Three generations: I, II, III

All three versions share the region structure above. They differ in how many networks model it, and how the controller decides which region it is in.



07 Which cluster am I in? The normalised distance

Each cluster C_{ij} is summarised by its mean μ_{ij} and its covariance Σ_{ij} . A current state is assigned to the cluster minimising

$$\text{NMD}^2_{ij}(z) = (z - \mu_{ij})^T N_{ij} (z - \mu_{ij})$$

A plain Euclidean distance would be wrong here, because these clusters are long thin slivers along the filament: a point can sit far from a cluster's centre and still be squarely inside it, or sit close to a centre but off the sliver entirely. The covariance term fixes that by measuring distance in units of the cluster's own spread, direction by direction.

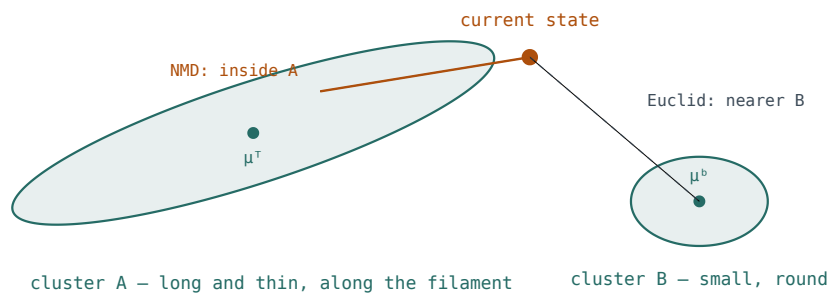


Fig 6. Why the covariance belongs in the distance. The state sits within cluster A's spread along the direction A actually extends in, so A's network is the one trained on states like this – even though B's centre is closer in plain distance.

What exactly N_{ij} should be is the one place where the original paper is silent: it calls it the "normalised covariance matrix" and leaves it there. The replication measured four readings on the Lorenz section, three trainings each, same 100 initial conditions:

READING OF N	REACH (STEPS)	SPREAD	RETENTION
unit determinant $(\Sigma/\det\Sigma^{1/N})^{-1}$	4.39	± 0.43	0.97
unit trace	4.26	± 0.23	0.96
plain Mahalanobis Σ^{-1}	5.18	± 1.30	0.61
variances only (diagonal)	5.17	± 1.02	0.67
no targeting at all	18.4	–	–

The two readings that strip out a cluster's *size* while keeping its *shape* win on all three counts, and they win most clearly on retention: the size-keeping readings hold the captured orbit only two thirds of the time. That is a real result about the method, not a tuning detail — and it is the kind of thing a replication can settle even though it cannot recover what the original authors actually did.

08 What each network actually computes

Every cluster carries one small radial basis function network. It maps a state to a *normalised* parameter perturbation — each output scaled so that ± 1 is the edge of the allowed range:

$$\Delta \hat{p}(z) = W^T [\varphi_1(z) \dots \varphi_k(z) 1]^T \quad \text{with} \quad \varphi_k(z) = \exp(-\|z - c_k\|^2 / 2s_k^2)$$

Three consequences worth noticing:

- **Admissibility becomes a box test.** "Is this parameter allowed?" is just "is every output within ± 1 ?" — which is also how a network says *I don't recognise this state*.
- **Training is a linear solve.** Centres come from k-means, widths from centre spacing, and the output weights from regularised least squares in closed form. No back-propagation, no epochs; a whole model trains in seconds.
- **Local, not global.** Gaussian bumps decay away from their centres, so a network says little about states far from its own cluster — which is exactly the behaviour you want when ten of them are competing.

09 The loop, and one real capture

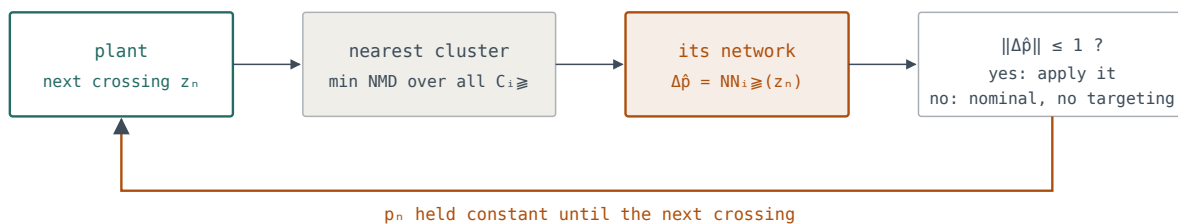


Fig 7. One pass per crossing. Notice what is *not* in the loop: no model of the plant, no state estimator, no optimisation. A distance, a small matrix multiply, and a bounds check.

Here is the capture from Fig 1, step by step, as the code actually logged it — the state, its distance to the target, which region and cluster claimed it, and the perturbation applied as a fraction of the allowed range:

STEP	X	Z	$\ Z - Z^*\ $	REGION	CLUSTER	$\Delta\Sigma$	ΔP	ΔB
0	14.8055	40.9439	1.2826	T ₁	2	+0.322	−0.322	−0.291
1	14.5548	40.2193	0.5279	T ₁	2	+0.018	+0.512	+0.123
2	14.2867	40.2052	0.4198	T ₁	2	+0.148	+0.467	+0.214
3	14.0810	39.7236	0.1825	—	—	inside the target region		

Two things in that table are worth pausing on. First, the perturbations are *small* — never more than about half the allowed budget, and mostly far less. Second, the same region claims the state three times running rather than stepping neatly $T_2 \rightarrow T_1 \rightarrow T_0$. That is honest behaviour, not a bug: the region label is a statement about what is reachable, and a chaotic map can leave you one step from home for several steps in a row while the controller nudges you onto the right approach.

10 Does it work, and where does it break?

LORENZ, REAL COORDINATES	ORIGINAL PAPER	REPLICATION
target z^* on the section	14.2387, 39.7934	14.2522, 39.7867
reaching time, no targeting	15.09	15.5
reaching time, ECR-II	5.9 – 6.7	4.5 – 6.0
reaching time, ECR-III	6.4 – 6.8	4.1 – 5.3

Three to four times faster than waiting, on a system where waiting is the only alternative if you refuse to model the plant. Now the caveats, which matter more than the headline:

- **The gain saturates.** Going from one control region to five buys much less than going from none to one. On a map that returns near the target every dozen crossings, there simply isn't room for a deep hierarchy.
- **Delay coordinates target but don't hold.** When the controller sees only a scalar measurement embedded with delays, reaching time still improves — but retention after capture collapses from 0.96 to about 0.2. The reason is structural: a delay vector is partly built from measurements taken while the *previous* parameter was applied, so the observed map isn't a function of the current state and parameter alone.
- **Deterministic feedback can deadlock.** A trajectory can lock into a *controlled* periodic orbit that never enters the target region. A single

nominal step after a run of fruitless controlled ones breaks it; so does measurement noise, which is why the noisy columns in the original tables sometimes look better than the noiseless ones.

- **The clustering is crude** — a radius and a chaining rule — and the original authors say so themselves, noting that a better algorithm would improve reaching time at the cost of training time.
-

11 The shortest version

1. Reduce the flow to a map by watching one plane. One crossing = one decision.
2. Run the plant with random admissible parameter nudges and record every transition.
3. Call T_0 the states that can be *kept* near the target, and T_i the states that can be pushed into T_{i-1} in one nudge. Decide membership using the network trained for the previous region.
4. Split each region into clusters; give each cluster a mean, a covariance and its own small RBF network.
5. On line: find the cluster with the smallest normalised distance, use its network, apply the nudge if it is legal, otherwise do nothing.
6. Measure success as the number of crossings to reach the target region — and separately, whether you can hold it once you arrive.

Numbers on this page come from a from-scratch MATLAB/Octave replication of the ECR method applied to the Lorenz system: 6000 recorded transitions, five control regions, 100 initial conditions, RK4 with Newton-refined section crossings.
Source: S. Iplikci & Y. Denizhan, *An improved neural network based targeting method for chaotic dynamics*, Chaos, Solitons & Fractals 17 (2003) 523–529.